



Experiment with vLLM on Kubernetes

Using MicroShift, Podman Desktop, and KServe

Alexon Oliveira

Senior Principal Technical Account Manager

The Inference Dilemma

Why Serving LLM is Hard



Massive Resources

Large models require gigabytes of VRAM. High-end GPUs (A100s/H100s) are scarce and expensive.



Complex State

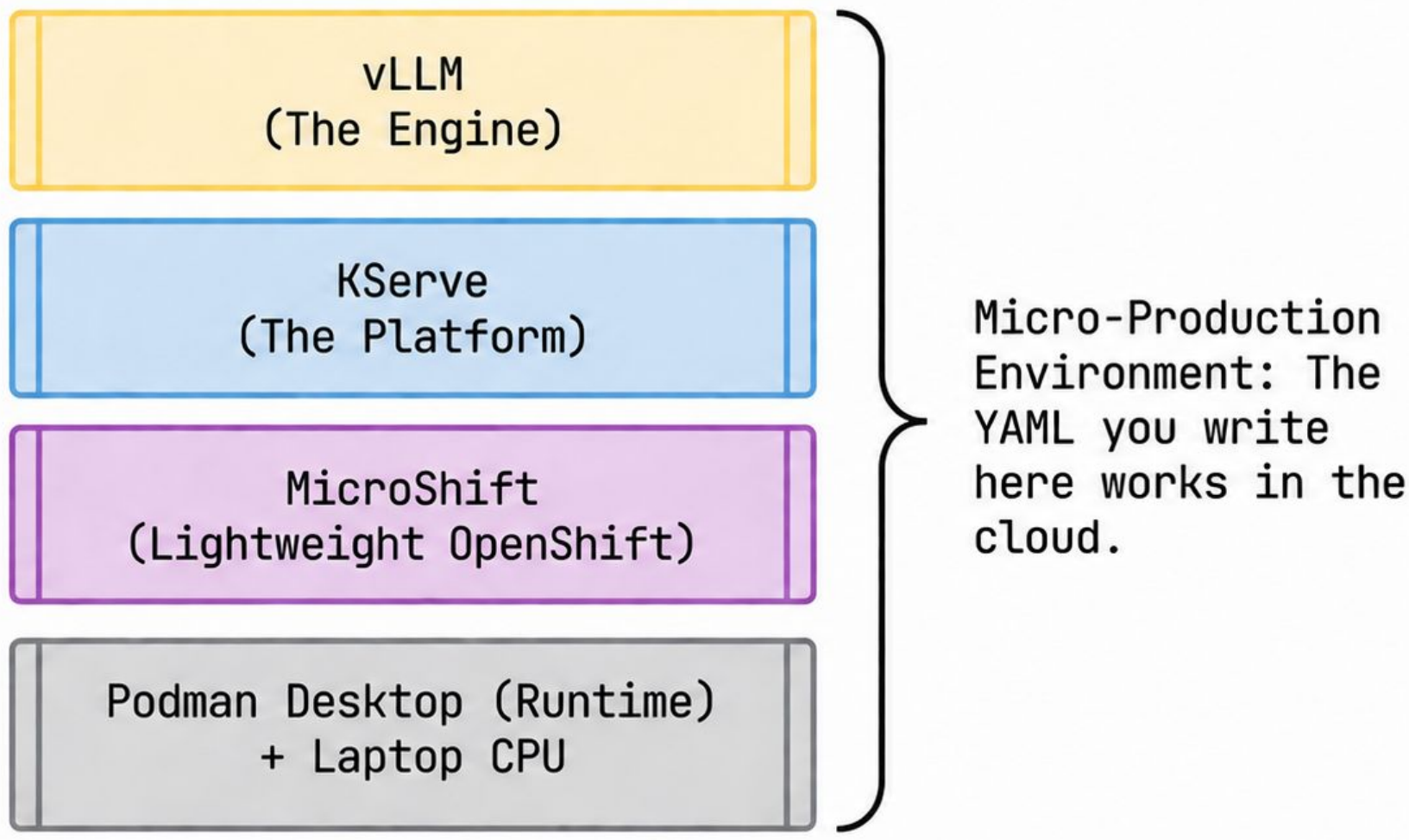
Unlike stateless microservices, LLMs manage 'KV Cache' for every user session, making load balancing difficult.



The Scaling Trap

Traffic spikes require furious scaling without dropping requests, but models take minutes to load into memory.

Meet Your Local Cloud Stack





The Engine

Why vLLM?



PagedAttention: Treats GPU/CPU memory like OS virtual memory. Eliminates fragmentation.



Continuous Batching: Processes incoming requests immediately rather than waiting for batch windows.

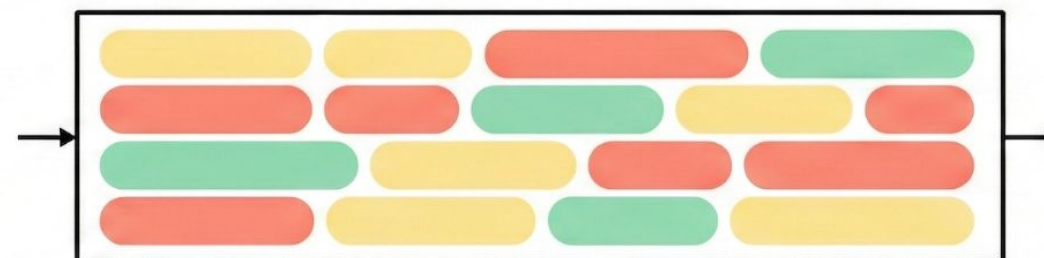


Performance: Up to 24x higher throughput than standard Transformers.

Individual Requests (Gappy Flow)



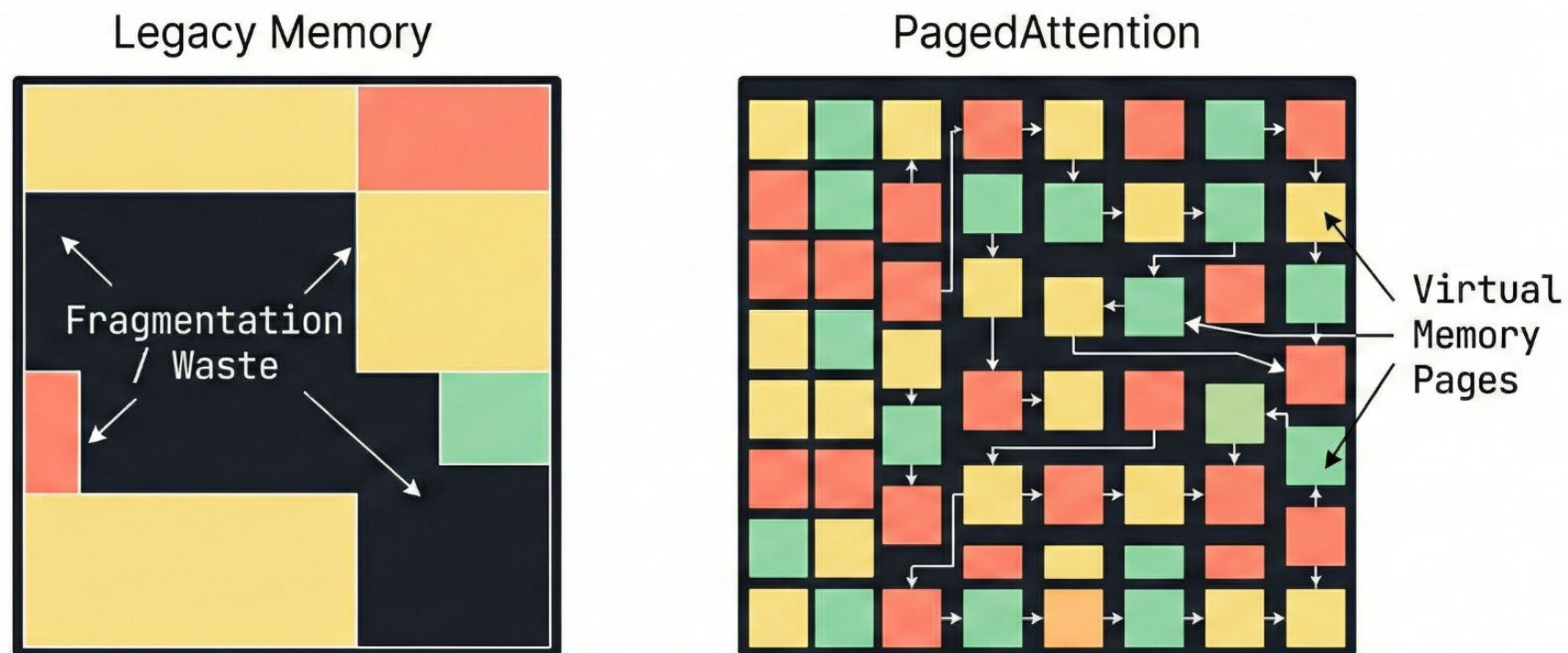
Continuous Batching (Efficient Pipeline)





Under The Hood

The Power of PagedAttention



Splits KV Cache into blocks stored anywhere in memory.
Result: Near-zero waste and higher concurrency.



The Platform

Why KServe?



Standardized Protocol

Implements V2 Inference Protocol
(OpenAI compatible API).



Serverless Capabilities

Native Scale-to-Zero and Autoscaling
based on traffic.



Pluggable Runtimes

Switch backends (Triton, TGI, vLLM)
without rewriting infrastructure code.

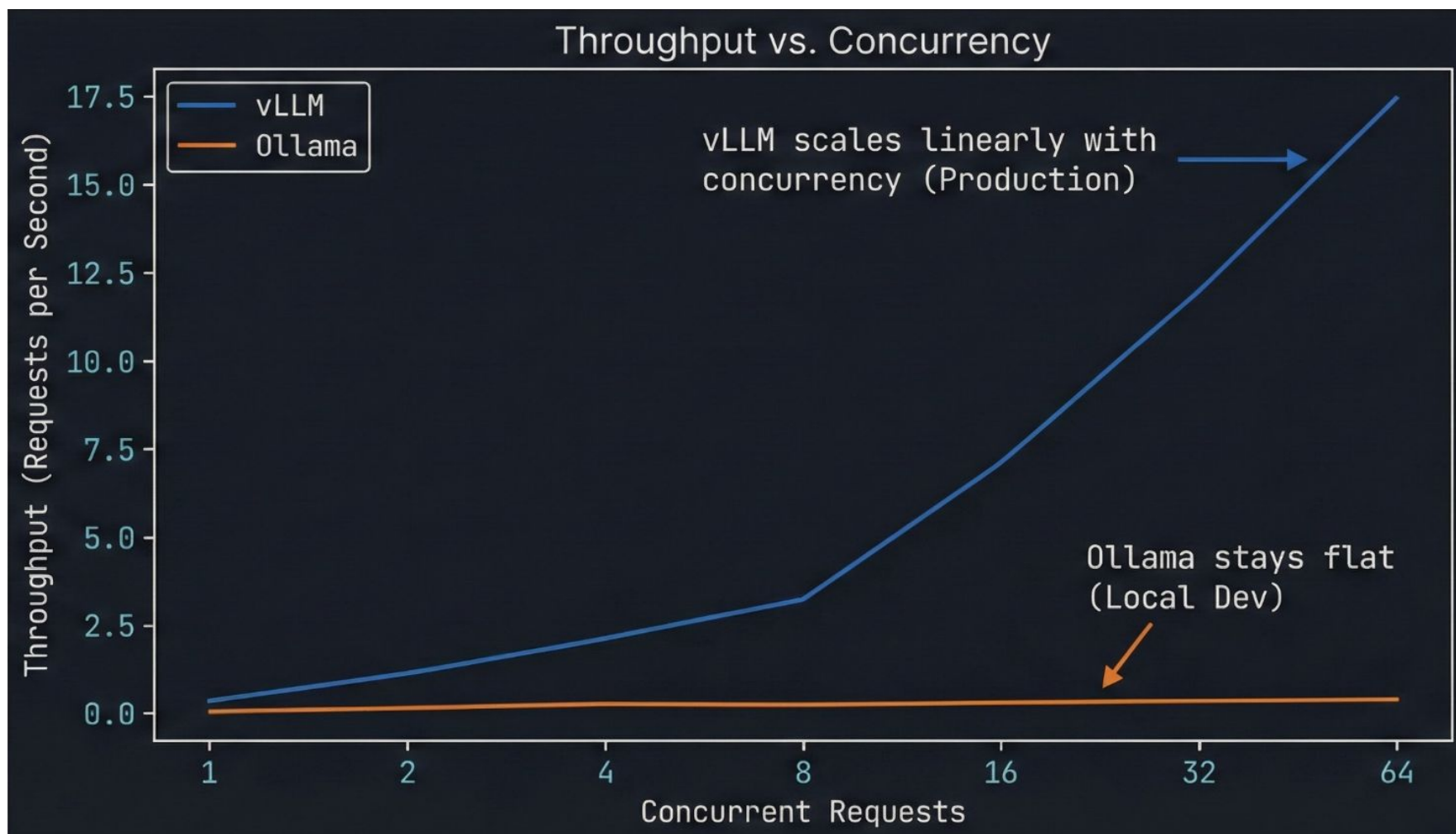


Abstraction

Manages networking, Ingress, Istio,
and revision rollouts automatically.

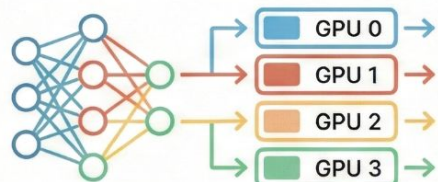
Benchmarks

vLLM vs Ollama



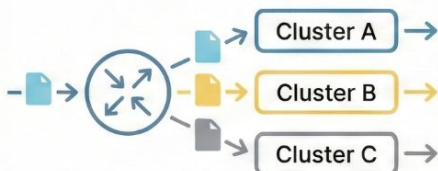
Extending to Production

Advanced Scaling



Tensor Parallelism (TP)

Split massive models (e.g., Llama-3-70B) across multiple GPUs.



Distributed Inference (llm-d)

Intelligent request routing across clusters.



Speculative Decoding

Use small draft models to predict tokens for big models.

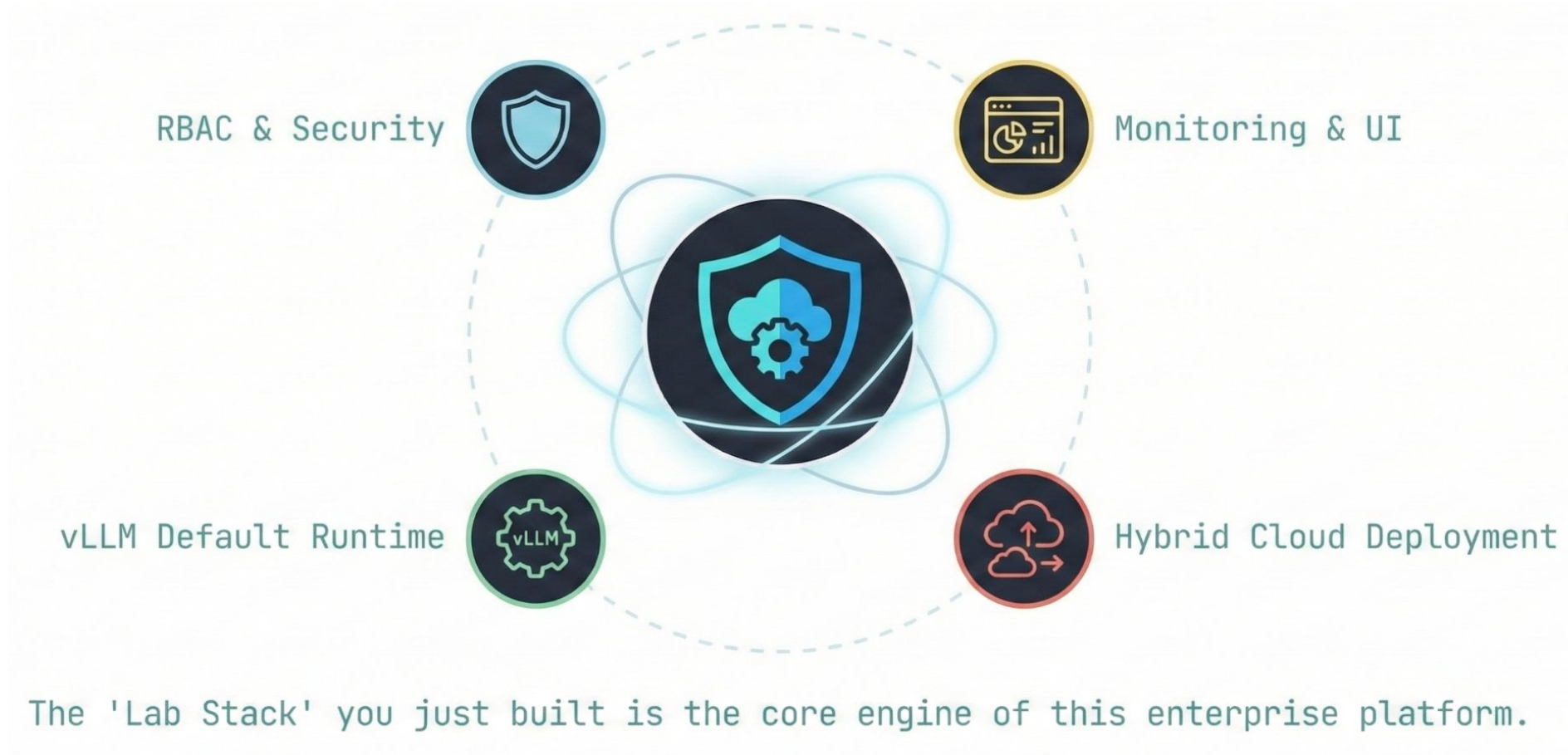


Hardware Agnostic

NVIDIA, AMD, Intel Gaudi, and AWS Inferentia.

The Enterprise Context

Red Hat OpenShift AI



The Lab Setup

CPU-Only Inference

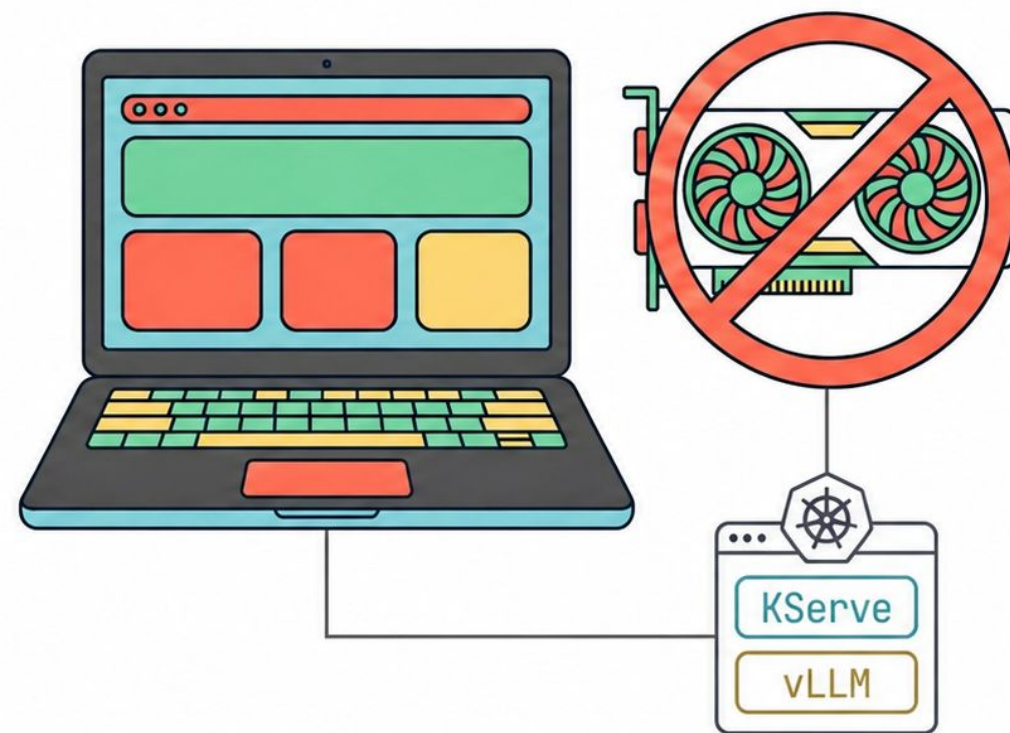
No A100? No Problem.

Constraint: Standard Laptop CPU

Model: google/gemma-3-270m

Goal: Validate KServe + vLLM pipeline

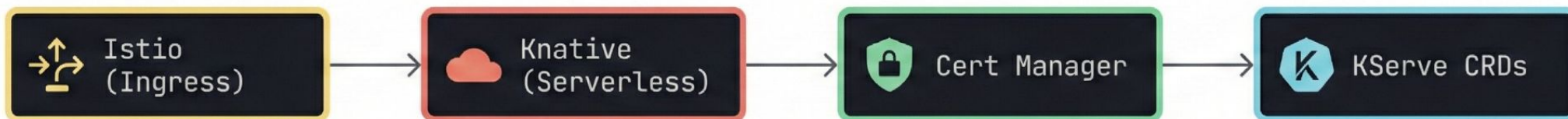
Prerequisites: Podman Desktop, MicroShift, Hugging Face Token



Step 1

Installing the KServe Stack

```
1 curl -s 'https://raw.githubusercontent.com/kserve/kserve/release-0.15/hack  
2   /quick_install.sh' | bash  
3
```



Step 2

The InferenceService (The Critical Config)

```
1 apiVersion: serving.kserve.io/v1beta1
2 kind: InferenceService
3 metadata:
4   name: gemma-3-270m
5 spec:
6   predictor:
7     image: kserve/huggingfaceserver:latest
8     args:
9       - --model_name=gemma-3-270m
10    env:
11      - name: VLLM_ATTENTION_BACKEND
12        value: 'CPU_ATTN'
13    resources:
14      limits:
15        cpu: '4'
16        memory: '8Gi'
```

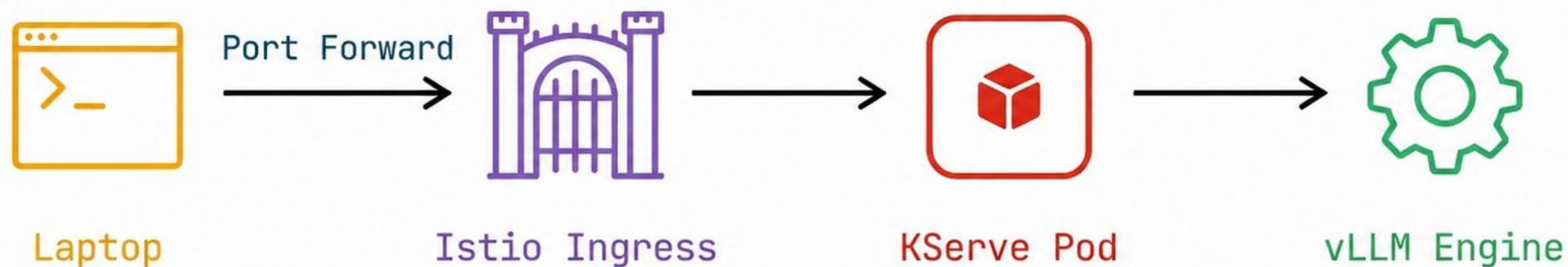
CRITICAL: Unlocks
local testing
without GPU

Sets bounds for
laptop hardware

Uses standard
HuggingFace
Server runtime

Step 3

Connecting the Pipes



```
1 # Check Port Forward
2 oc port-forward svc/istio-ingressgateway ...
3 # Identify Host
4 export SERVICE_HOSTNAME=$(oc get inferencservice ...)
```

Running Inference

Live Results

```
$ ./03-run-inference.sh
Request: 'What is the capital of France?'
{
  "id": "cmpl-8912...",
  "object": "text_completion",
  "choices": [
    {
      "text": "The capital of France is Paris.",
      "index": 0,
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 7,
    "completion_tokens": 8,
    "total_tokens": 15
  }
}
```

Success: Kubernetes-native AI inference running on CPU.

Demo

Q Search

Dashboard

Dashboard

Containers

Pods

Images

Volumes

Networks

Extensions

Minikube

All

Remote Containers

Kubernetes

Podman Desktop

Quickstart

Contexts

Subscriptions

SSH

LightSpeed

Accounts

Settings

> Explore Features

> Learning Center

Podman v4.9.2

Podman

Website

Installation guide

Docker compatibility guide

Join the community

Developer Sandbox

OpenShift Local v2.60.3 (OpenShift Local)

INITIALIZED BUT NOT READY

To start working with containers, OpenShift Local needs to be initialized.

Initialize and start

Website

Installation guide

Obtain pull-secret

Troubleshooting

Repository

Podman

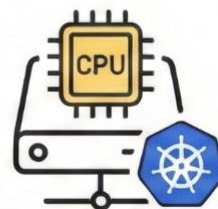
No context

v1.26.2

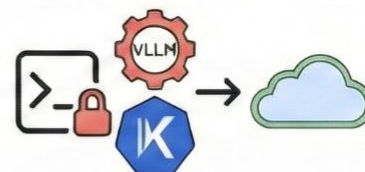
Key Takeaways

Next Steps

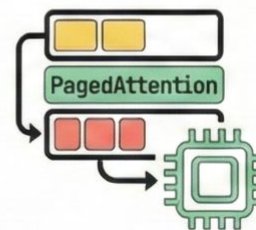
1. **Start Local:** Use 'CPU_ATTN' to learn K8s patterns without cloud bills.



2. **Use Production Tools:** vLLM + KServe locally = painless migration.



3. **Optimize:** Understand PagedAttention to maximize hardware.



Clone the Repo
(Scripts 01-03)





Thank you

 linkedin.com/company/red-hat

 facebook.com/redhat

 instagram.com/red_hat

 x.com/RedHatSummit